Data Structures
Lab Assignment 1

Linear data structures and image processing

Fall 2026-2027

Kaito Snezhko - Student ID: 52407400

Ana Gasperov - Student ID: 116931401

Index

1. Analysis

1.1 Data flow

1.2 ADT specifications

2. Design

2.1 Memory representation and operations

2.2 Use-case diagram

2.3 Class diagram

2.4 Classes and significant methods

2.5 Program behaviour

3. Implementation

3.1 Source files and relations

3.2 Difficult sections

4. Review

4.1 Running time

4.2 Possible enhancements

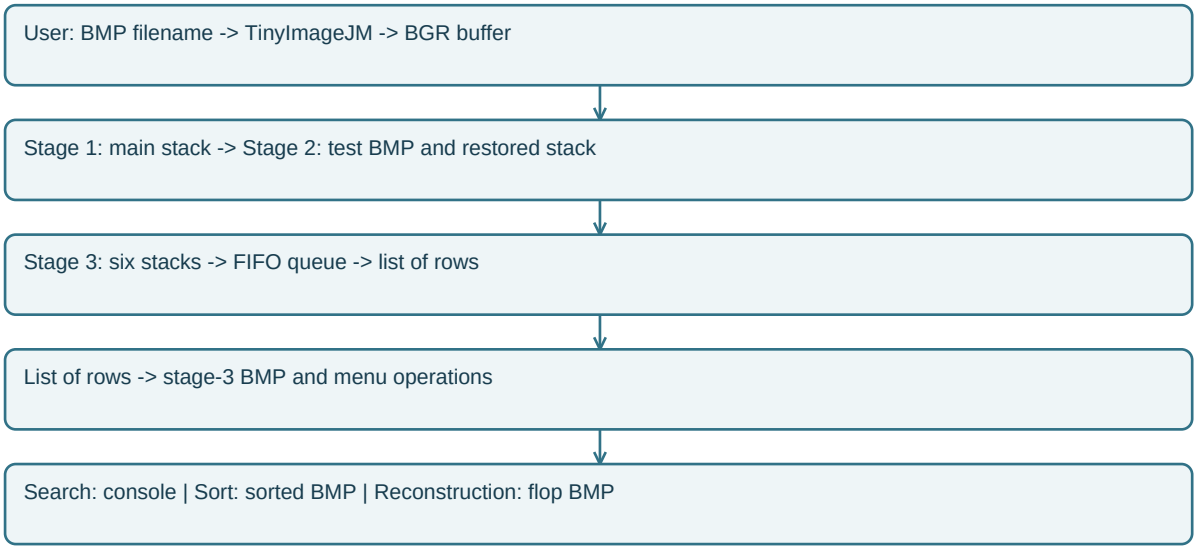
4.3 Dynamic and static data structures

5. References

1. Analysis

1.1 Data flow

The user enters a BMP filename. Core requests disk access through TinyImageJM, obtains width W, height H and a pointer to the library-owned image buffer, and interprets every three bytes as blue, green and red. A Pixel stores RGB values and a link, without coordinates. Coordinates are inferred from traversal order. Let N = W x H.



Automatic processing and menu operations

Stage 1: Push all N pixels in buffer order. The first file pixel becomes the bottom of pixelStack and the last becomes its top. Width and height remain in Core, separate from pixel nodes.

Stage 2: Pop into buffer positions N-1 down to 0 and also push each value onto a helper stack. Move all helper values back to pixelStack. The test image preserves the input pixel order and the main stack regains its original content and order.

Stage 3: Number successive main-stack pops from 1. Apply the first matching distribution rule to choose one of six stacks. Fill a queue using the required alternating selection rule, then dequeue W pixels into each of H rows. The resulting linked image is the persistent stage-3 representation.

Stage 4: Search the persistent rows, sort copies of each row, reconstruct the original image from the persistent order and mirror it, or exit. Search writes coordinates to the console; image operations write BMP files. Menu operations leave the stored rows unchanged.

Assumptions and design decisions

The intended input is an ordinary uncompressed, positive-height 24-bit BMP. Its buffer is bottom-row-first and contains no row padding; the library handles disk padding. The overlapping divisibility rules use their listed priority. File paths are read with operator >>, so filenames containing spaces are not supported. Search coordinates use a top-left origin: x starts at 0 and displayed y = H-1-rowIndex. Sorting is ascending by exactly one component and preserves the RGB tuple of each pixel.

1. Analysis

1.2 ADT specifications

1.2.1 Pixel

Pixel combines a colour value with the next pointer used by all linked pixel collections. Pixel() creates black (0,0,0) with next = nullptr. Pixel(unsigned char red, unsigned char green, unsigned char blue) creates the given colour and a null link. getRed(), getGreen() and getBlue() return unsigned char; getComponent(char) returns an int for R or G, and blue otherwise. The controller validates R/G/B before sorting. getNext() returns Pixel*; setNext(Pixel*) returns void. These operations take O(1) time.

1.2.2 StackDS

Adaptation: The stack stores only pixel colours and has no count field. A top pointer is enough for the required LIFO operations. The same specification applies to the stage-1 stack, stage-2 helper, six stage-3 stacks and both arrays of reconstruction stacks. Every push allocates a fresh node containing RGB only; input links are not reused.

StackDS(): No arguments; initializes an empty stack. ~StackDS(): No arguments or return value; repeatedly pops and deletes all remaining nodes.

push(Pixel p_pixel) -> void: Insert a colour copy at the top. If the old abstract sequence is [a,b,...] from top down, it becomes [p_pixel,a,b,...].

pop() -> Pixel: Remove and delete the top node, returning its colour with a null next link. For an empty stack, return a default black pixel and leave the stack empty. Normal callers avoid underflow through emptiness checks or known element counts.

top() -> Pixel: Return the top colour without changing the stack, or black if empty. isEmpty() -> bool: Return topElem == nullptr.

Invariant: topElem is null exactly when empty; otherwise following next reaches each owned node once and terminates at null. No cycle or shared ownership is intended. Each operation is O(1), excluding destruction, which is O(k) for k nodes.

1.2.3 QueueDS

Adaptation: Pixel nodes are linked from front to rear. Both endpoint pointers and a size field support constant-time insertion, removal and count reporting. The same specification applies to the stage-3 queue and the local reconstructionQueue.

QueueDS(): Initialize frontElem = rearElem = nullptr and size = 0. ~QueueDS(): Dequeue until empty, deleting all nodes; O(k).

enqueue(Pixel p_pixel) -> void: Append a new RGB node at the rear. If empty, set frontElem to the new node as well; update rearElem and increment size.

dequeue() -> Pixel: Remove and delete the front node and return its colour. Decrement size and clear rearElem if the last node was removed. Empty removal returns black and leaves size unchanged.

isEmpty() -> bool: Return frontElem == nullptr. getSize() -> int: Return the stored size. All four operations are O(1).

Invariant: size equals the number of reachable nodes. Empty means both endpoints are null and size is zero; otherwise rearElem is the last reachable node and rearElem->next is null.

1. Analysis

1.2 ADT specifications

1.2.4 RowListDS

Adaptation: A singly linked list has first and last pointers and a size field. The last pointer makes append O(1). Each row also carries nextRow, allowing the outer image list to link existing row objects without an additional wrapper-node class. A row owns its pixels but does not own the next row. The persistent image rows and each temporary sortedRow use the same specification.

RowListDS(): Initialize first = last = nullptr, size = 0 and nextRow = nullptr. ~RowListDS(): Traverse the pixel chain and delete its nodes; do not delete nextRow. Cost O(k).

insertEnd(Pixel p_pixel) -> void: Allocate a new colour node and append it. Initialize both endpoints when empty, otherwise link after last. Increment size; O(1).

insertSorted(Pixel p_pixel, char p_component) -> void: Assuming the current row is already sorted by a valid R/G/B component, insert a new colour node after all existing nodes with equal component value. Handle empty, new-minimum and value-at-least-last cases directly; otherwise scan to the insertion point. Update endpoints and size. O(1) for endpoint cases, O(k) worst case.

getFirst() -> Pixel*: Return the first node or nullptr. getSize() -> int: Return the count. isEmpty() -> bool: Test first == nullptr.

getNextRow() -> RowListDS*: Return the adjacent row or nullptr. setNextRow(RowListDS* p_nextRow) -> void: Replace the adjacent-row link. All accessors and this link setter are O(1). Returned pointers are borrowed; ownership stays with the collections.

Invariant: first and last are both null for an empty row; otherwise last terminates the pixel chain. size equals the reachable pixel count. The nextRow chain must remain acyclic. During construction, row sizes may be partial; after stage 3 each persistent row has exactly W pixels.

1.2.5 ImageListDS

Adaptation: The outer list directly owns RowListDS objects, using their nextRow links. firstRow and lastRow support O(1) append; numRows reports the number of rows. This is the list-of-lists representation required by the assignment.

ImageListDS(): Set firstRow = lastRow = nullptr and numRows = 0. ~ImageListDS(): Delete every row in sequence; each row destructor deletes its pixels. For H rows and N pixels the cost is O(H+N).

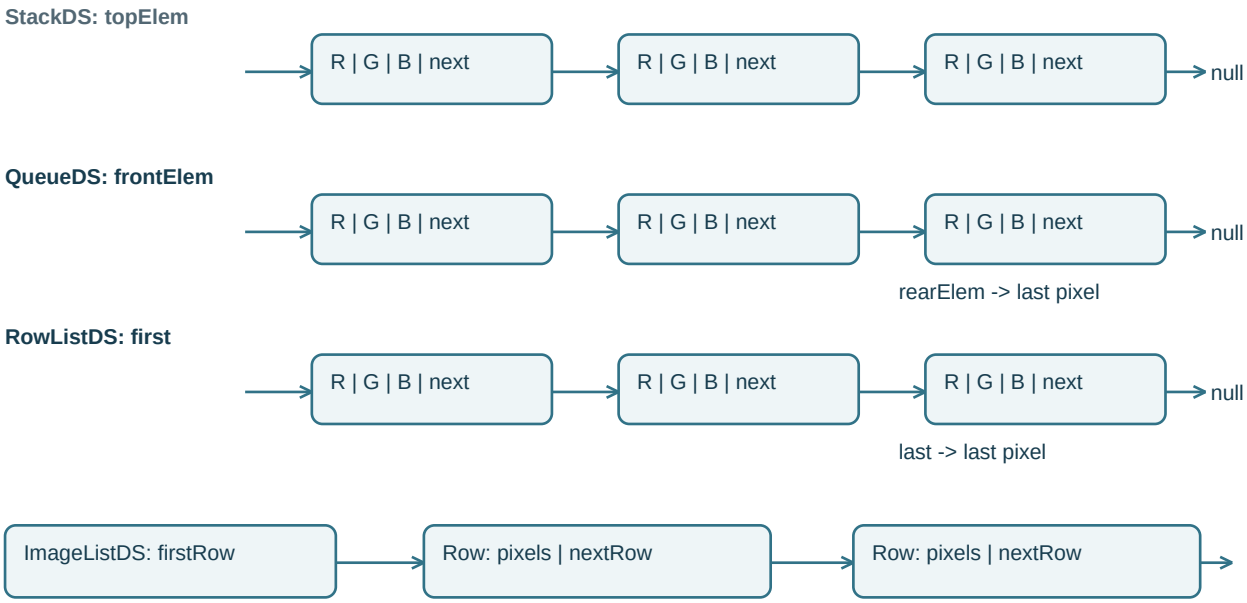
addRow(RowListDS* p_row) -> void: Append the row and assume ownership. The caller must supply a non-null, separately allocated, unowned row whose nextRow is null. Set both endpoints if empty, otherwise attach after lastRow; increment numRows. O(1).

getFirstRow() -> RowListDS*: Return a borrowed pointer to the first row or nullptr. getNumRows() -> int: Return numRows. Both are O(1).

Invariant: numRows equals the number of reachable row objects. lastRow is the final row, with a null nextRow. After stage 3, numRows = H and the total number of pixels is N. The structure is traversed without mutation by all menu operations.

2. Design

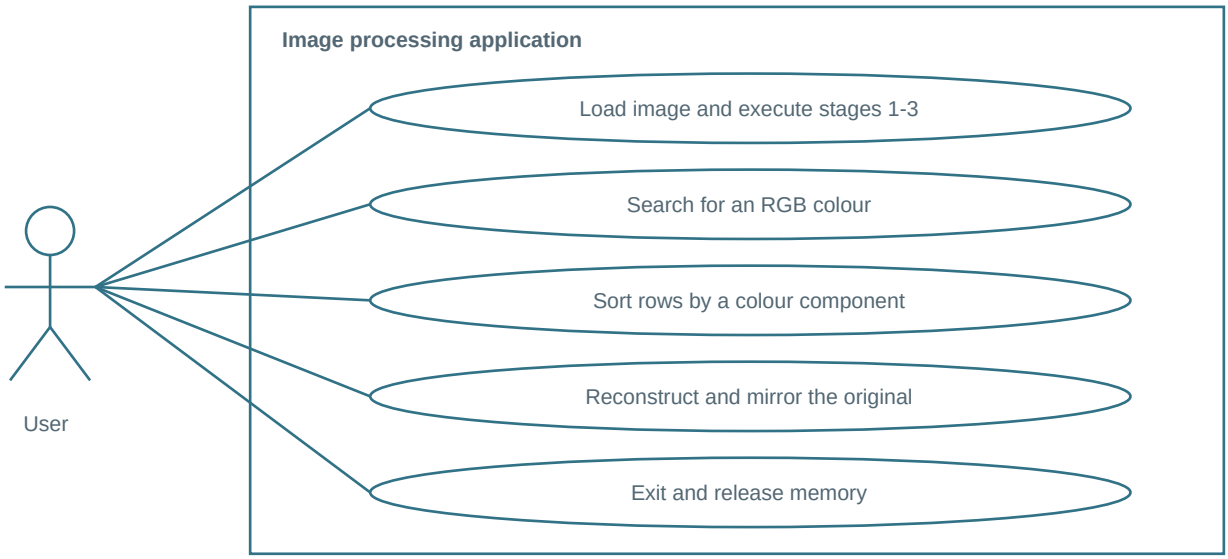
2.1 Memory representation and operations



Pixel nodes store the three colour components and a next pointer. StackDS owns the chain beginning at topElem. QueueDS owns a chain between frontElem and rearElem. RowListDS owns the chain between first and last. ImageListDS owns rows linked through nextRow; each row deletes its own pixels.

Stack push redirects the new node to the previous top; pop advances top before deleting the old node. Queue enqueue links through rear; dequeue advances front and resets rear on the last removal. Row insertion redirects neighbouring links and adjusts first or last when needed. The outer list links row objects through nextRow; each row independently points to its pixel chain.

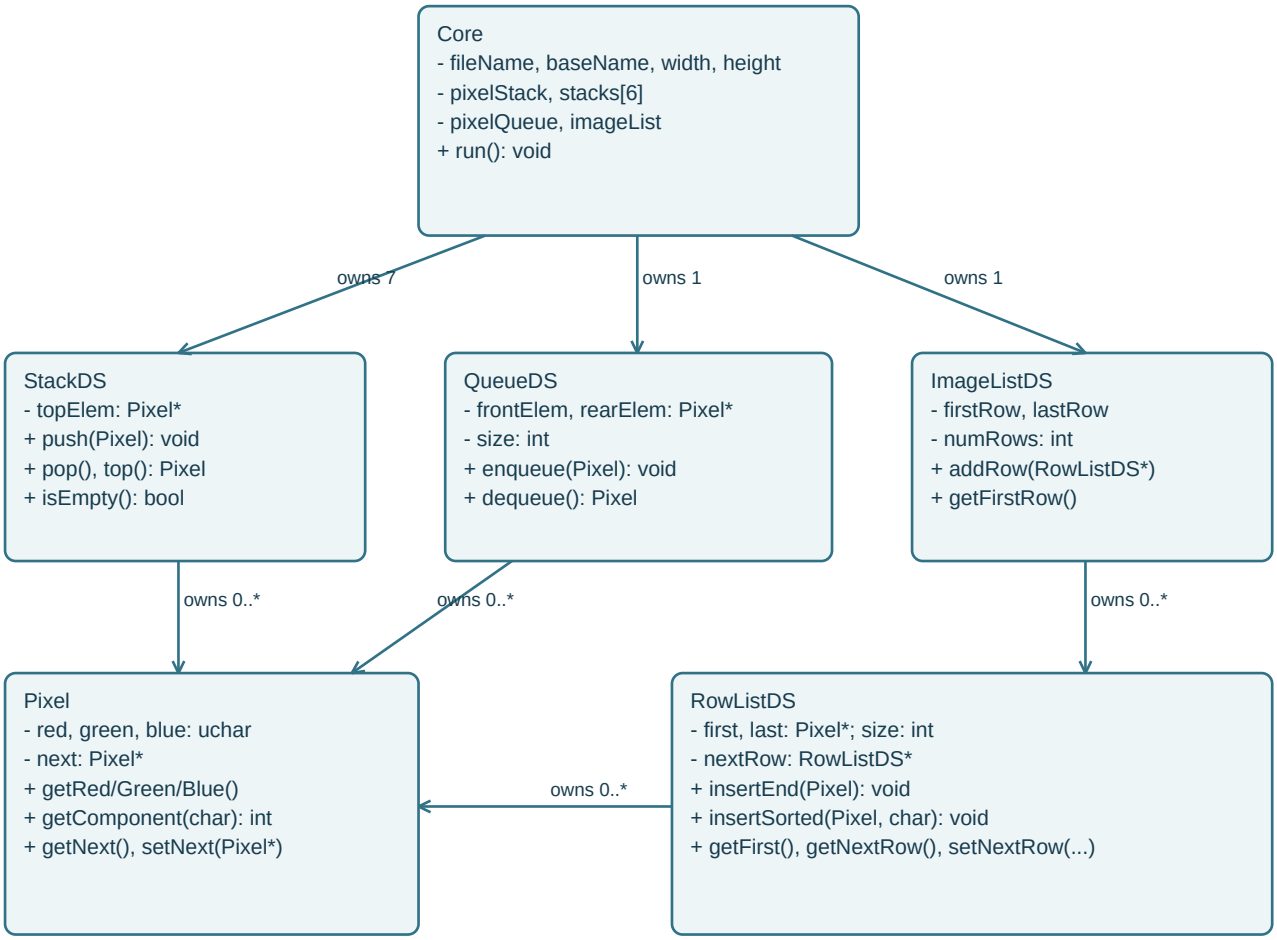
2.2 Use-case diagram



After loading an image, the program executes the first three stages automatically. The user can then repeat search, sort and reconstruction operations until choosing Exit.

2. Design

2.3 Class diagram



Arrows indicate ownership; labels give multiplicities. Private members use (-) and public operations use (+). Pixel links to the next Pixel; RowListDS links to the next RowListDS. Core also uses TinyImageJM to load and save images.

2.4 Classes and significant methods

Core: Owns persistent collections, validates interactive input, coordinates stages and manages output filenames. run() calls stages in order only after successful loading. chooseStack(int) implements rule precedence; fillQueue() implements interleaving; fillListOfLists() creates row ownership; writeRowToBytes() serializes RGB nodes as BGR and returns the next free buffer index.

Pixel and collections: Pixel stores colours and links. StackDS and QueueDS implement their standard LIFO/FIFO behaviours. RowListDS supports append and stable ordered insertion. ImageListDS owns and links rows. The data-structure implementations contain neither console I/O nor file access.

TinyImageJM: Loads and saves BMPs, exposes dimensions and its internal buffer, and frees its buffer in its destructor. Core reloads the source to obtain an allocated output image buffer and then overwrites pixel bytes. It does not replace the linked structures with an STL container.

Temporary sorted rows and reconstruction structures are destroyed when their scope ends. On exit, the Core members release the persistent structures. ImageListDS deletes every row, and each RowListDS destructor deletes its pixels.

2. Design

2.5 Program behaviour

The program requests a BMP filename. A failed load produces an error and repeats the request; entering q ends the program. After a successful load, it prints the image dimensions and the initial stack information.

Stage 2 creates the test image and reports how many pixels were removed and restored. Stage 3 reports the six stack counts, queue size, completion of stack draining, and dimensions of the resulting list of lists. The queue is empty after row construction.

Stages 1-3 run automatically. Option 1 asks for red, green and blue values in 0-255 and prints all exact matches as (x,y), ten per line. Search refers to the stage-3 rearrangement, not the original image. Option 2 accepts R/G/B (lowercase is accepted) and saves a separately sorted image. Option 3 reconstructs and horizontally mirrors the original. Option 4 exits.

For input small.bmp, output files are small_test.bmp, small_stage3.bmp, small_sorted_R.bmp (or G/B), and small_flop.bmp. The same suffix rules apply to other filenames, with output beside the specified input path. Existing output files with these names are removed before saving, so repeating an operation replaces its earlier result.

Search

The user enters integer RGB components between 0 and 255. The program compares all three components of every stored pixel and prints matching (x,y) coordinates, with ten coordinates per line, followed by the total number of matches. Coordinates are measured from the top-left of the stage-3 image.

Sorting

The user chooses R, G or B. Each row is copied into a new list using stable insertion sort in ascending order. The program prints a sample of the first row before and after sorting, saves the sorted image, and destroys the temporary row lists. The stored stage-3 image remains unchanged.

Reconstruction and exit

Reconstruction reverses the stage-3 rearrangement to recover the original pixel order, then mirrors each original row horizontally. The result is saved with the _flop suffix. Exit ends the menu loop and releases all allocated data-structure nodes.

Input data

Three sample images are included: small.bmp (800 x 800), mid.bmp (1108 x 1108), and big.bmp (2000 x 1334). They provide different input sizes for executing the four stages and the menu operations.

3. Implementation

3.1 Source files and relations

```
main.cpp -> Core.hpp / Core.cpp
              |-> StackDS.hpp / StackDS.cpp -> Pixel.hpp / Pixel.cpp
              |-> QueueDS.hpp / QueueDS.cpp -> Pixel.hpp
              |-> ImageListDS.hpp / ImageListDS.cpp
              |       -> RowListDS.hpp / RowListDS.cpp -> Pixel.hpp
              |-> peque-image-manager/TinyImageJM.hpp / .cpp

DS-LabAssignment1.workspace -> Assignment1/Assignment1.project
```

Each implementation file includes its corresponding header. Core coordinates the collections and image library. ImageListDS depends on RowListDS, and all pixel collections depend on Pixel. Console and file operations are contained in Core and TinyImageJM, outside the data-structure implementations.

3.2 Difficult sections

Stack distribution and queue generation

```
chooseStack(k): first matching rule wins
  k == 1 or k divisible by 2 -> S1
  else divisible by 3       -> S2
  else divisible by 5       -> S3
  else divisible by 7       -> S4
  else divisible by 11      -> S5
  else                      -> S6

For queue position j = 1..N:
  j == 1 or j even: pop S1 if available
  otherwise: pop first nonempty stack in S6, S5, ..., S1
```

Rules are evaluated in their listed order. The pixel counter starts at 1 and counts removals from the main stack. Queue positions also start at 1. The six-stack search selects the first nonempty stack in decreasing stack-number order.

Stage 2: Writing popped pixels backwards compensates for LIFO reversal. Moving the helper stack back performs a second reversal and restores the original main stack.

Sorting: For each persistent row, start an empty sortedRow and call insertSorted for each pixel. Write this temporary row and destroy it before processing the next one. The middle insertion scan passes equal keys; the endpoint append test also accepts equality. These decisions make the ascending insertion sort stable.

Reconstruction: Count the original membership of all six stacks with chooseStack. Copy the persistent row traversal into a queue. Replay the forward queue-source selection using the remaining counts, pushing dequeued pixels onto reverseOrder[source]. Reverse each of these stacks into reconstructedStacks to recover the stack contents that existed before queue filling. Iterate pixelNumber from N down to 1 and pop reconstructedStacks[chooseStack(pixelNumber)-1]. The original file index is N-pixelNumber. Write each recovered pixel at row*W + (W-1-column). This restores the original order and mirrors horizontally without changing the stored image.

Library adaptation: TinyImageJM.cpp recalculates 24-bit row padding after reading width, correcting a calculation previously made while width was zero. For 24-bit BMPs the number of padding bytes is width % 4. Core deals only with the compact, padding-free buffer.

4. Review

4.1 Running time

Let W and H be positive image dimensions, N = W x H, k the current length of a collection and M the number of search matches. Bounds assume successful allocation and count image reads/writes as linear in file size; console output adds a cost proportional to printed text.

Stage 1: Read the BMP and push N nodes: O(N) time and O(N) storage. Stage 2: Pop, serialize and restore each pixel a constant number of times: O(N) time and O(N) total storage, including the helper stack and image buffer.

Stage 3: Distribution, interleaving, queue draining and output serialization are each O(N). Creating H rows adds O(H), so the total is O(N+H), equivalent to O(N) when W >= 1. The six-way tests have constant cost. The image list occupies O(N+H); transfers briefly hold a constant number of representations, leaving peak space O(N+H).

Search: O(N) traversal plus O(M) coordinate output, with O(1) auxiliary collection storage. Matches are printed immediately rather than accumulated.

Sort: Worst-case O(H x W^2), because each of W ordered insertions can scan a row. With endpoint insertions throughout, including already ascending or descending component order, the implemented shortcuts give O(N) best-case time. Temporary linked storage is O(W), in addition to the persistent list and O(N) image buffer. Stable ties preserve input order within each row.

Reconstruction and mirroring: Counts, queue copying, replay, stack reversal and output writing each take O(N); total O(N). Auxiliary queues/stacks store O(N) pixel nodes. Source rows remain present throughout. Exit: Destruction is O(N+H), normally dominated by the persistent image list.

4.2 Possible enhancements

Memory usage and allocation overhead could be reduced by transferring existing pixel nodes between structures or using a node pool. Reconstruction could also reuse temporary storage to reduce the number of simultaneous pixel copies.

A stable linked merge sort would reduce the worst-case sorting time from O(H x W^2) to O(H x W log W). The current implementation uses insertion sort and its constant-time endpoint shortcuts.

Input handling could accept filenames containing spaces and validate BMP dimensions, format and read results more thoroughly. Every output reload and save could be checked explicitly. Collection classes could prevent shallow copying of owned pointers, and empty removal operations could return a success indicator separately from the pixel value.

4. Review

4.3 Dynamic and static data structures

Stacks: Dynamic nodes grow as pixels are read and naturally model transfers without a fixed image-capacity limit. They also incur one allocation per pushed value and pointer overhead. Since N is known after loading, a preallocated array stack would use less memory and improve locality; dynamic stacks are used to meet the pointer-based learning objective.

Queues: Front and rear links make FIFO transfers O(1) without shifting elements. A fixed-capacity circular array of N pixels could provide the same asymptotic operations with fewer allocations, but would require capacity management and would not demonstrate linked queue storage.

Rows and outer list: Linked lists allow insertion by redirecting links and independent ownership of rows. They still require linear scans to find an interior insertion point. A contiguous W-by-H image has better random access and locality; the nested linked representation instead demonstrates the required list-of-lists ADT. Coordinates are derived during traversal, saving redundant per-node coordinate fields.

Small static arrays: The six-stack arrays and six-element counters have compile-time fixed size because the assignment fixes the number of buckets. Their contents remain dynamic linked structures. The image library uses a contiguous byte buffer for disk conversion, complementing the linked representation.

5. References

[1] Data Structures. Laboratory Assignments, Fall 2026. 2026-2027-DS-LabAssignment-Fall.pdf. Assignment 1 and documentation requirements.

[2] TinyImageJM. Image-loading and image-saving library supplied with the assignment, included in peque-image-manager/TinyImageJM.hpp and TinyImageJM.cpp.